URDU-INK MOBILE: A GAN-ENHANCED REAL-TIME HANDWRITING RECOGNITION KEYBOARD

Waqar Ahmad

Isna Jadoon

Supervised By

Dr. QuaratulAin Safdar

Submitted for the partial fulfillment of BS Computer Science degree to the Faculty of Engineering & CS

DEPARTMENT OF COMPUTER SCIENCE

NATIONAL UNIVERSITY OF MODERN LANGUAGES, ISLAMABAD

November, 2026

ABSTRACT

Urdu is the language of more than 230 million people across the world, yet slow and unnatural phonetic or fixed-layout keyboards are used to write it on digital devices like smartphones. Our project presents *Urdu-Ink Mobile*, a real-time handwriting recognition keyboard that users can use to write Urdu naturally with their finger or stylus and then convert the handwritten word into editable Unicode text. Preexisting academic systems only recognize isolated characters or ligatures, while commercial offerings like Gboard offer very limited support for Nastaliq style handwriting. This project extends the half-form recognition pipeline of Safdar et al. (2022), which uses pre-classification and wavelet-feature to achieve an accuracy of 96.1% to recognize full-form words.

The system is organized in multiple stages. Raw handwriting data in the form of XY coordinates, time and pressure values was collected using a flutter based canvas. The raw data was then passed through a jitter removal, downsampling, interpolation and normalization pre-processing pipeline to prepare the dataset for model training. A GAN network was trained to on the collected dataset to generate synthetic samples to extend the dataset. A Neural Network was then trained on both the organic and synthetic samples to recognize handwritten Urdu words. The models were developed using Python libraries PyTorch and Keras and deployed using TenosrFlow lite.

The system was tested using unit, integration, black-box and performance testing on Infinix Hot 11 Play with MediaTek Helio G36 processor with 4 GB of RAM running on Android 11 and .[device model, processor, RAM and Android/iOS version]. Recognition accuracy was measured using Character Error Rate and Word Error Rate. The tests showed a CER of [value]% and a WER of [value]% on [test set description], with an average end-to-end latency of [value] ms per word, all of which met [or did not meet] the targets set in the non-functional requirements.

The current system is limited by the size of the dataset. It can lose accuracy on very complex and long words. Expanding the dataset to include more writers and samples could be possible future works.

CERTIFICATE

Final Approval

Dated: _____

It is certified that the project report titled **'Urdu-Ink Mobile: A GAN-Enhanced Real-Time Handwriting Recognition Keyboard'** submitted by **Waqar Ahmad** and **Isna Jadoon** for the partial fulfillment of the requirement of **"Bachelor's Degree in Computer Science"** is approved.

COMMITTEE

[Name of Dean] Signature: _____
Acting Dean Engineering & CS:

Dr. Noman Malik Signature: _____
HoD Computer Science:

[Name of Coordinator] Signature: _____
Project Coordinator:

Dr. QuaratulAin Safdar Signature: _____
Supervisor:

DECLARATION

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group members:

Name	**Signature**
Waqar Ahmad	_____
Isna Jadoon	_____

PLAGIARISM CERTIFICATE

This is to certify that the project entitled "**Urdu-Ink Mobile: A GAN-Enhanced Real-Time Handwriting Recognition Keyboard**", which is being submitted herewith for the award of the "**Degree of Bachelor's in Computer Science**". This is the result of the original work by **Waqar Ahmad** and **Isna Jadoon** under my supervision and guidance. The work embodied in this project has not been done earlier for the basis of award of any degree or compatible certificate or similar title of this for any other diploma/examining body or university to the best of my knowledge and belief.

Turnitin Originality Report
Processed on [dd-Mon-yyyy hh:mm PKT]
ID: [Turnitin ID]
Word Count: [count]

Similarity Index
[xx]%

Similarity by Source
Internet Sources: [xx]%
Publications: [xx]%
Student Papers: [xx]%

Date: [dd/mm/yyyy]

_____
Dr. QuaratulAin Safdar (Supervisor)

ACKNOWLEDGMENT

We express our sincere gratitude to our supervisor, Dr. QuaratUl Ain Safdar, for her guidance, patience and constructive feedback throughout this project. We also thank the volunteers who contributed their Urdu handwriting samples, and our families for their constant support.

TABLE OF CONTENTS

Chapter **Page**

CHAPTER 1: INTRODUCTION 1
1.1 Project Domain 2
1.2 Problem Identification 2
1.2.1 Proposed Solution 3
1.2.2 Objectives 3
1.2.3 Scope of the Project 3
1.3 Effectiveness / Usefulness of the System 3
1.4 Resource Requirement 4
1.4.1 Hardware Requirement 4
1.4.2 Software Requirement 4
1.4.3 Data Requirement 5
1.5 Report Organization 5

CHAPTER 2: BACKGROUND AND EXISTING SYSTEMS 6
2.1 Related Literature Review 7
2.2 Related Systems/Applications 8
2.3 Identified Problem from Existing Work 9
2.4 Selected Boundary for Proposed Solution 9

CHAPTER 3: SYSTEM REQUIREMENTS AND SPECIFICATIONS 10
3.1 System Specification 11
3.2 System Modules 11
3.2.1 Data Acquisition Module 11
3.2.2 Urdu Handwriting Corpus Module 11
3.2.3 Preprocessing Engine 12
3.2.4 GAN-Based Stroke Generation Module 12
3.2.5 Ligature Segmentation Module 12
3.2.6 Pre-Classifier Module 12
3.2.7 Hybrid Feature Extraction Module 12
3.2.8 Recognition Core 13
3.2.9 Language Model Layer 13
3.2.10 Real-Time Inference Engine 13
3.2.11 Evaluation and Analytics Module 13
3.3 Functional Requirements/Software Features 13
3.3.1 Stroke Capture (FR1) 13
3.3.2 Stroke Preprocessing (FR2) 13
3.3.3 GAN-Based Stroke Generation (FR3) 14
3.3.4 Segmentation and Pre-classification (FR4) 14
3.3.5 Handwriting Recognition (FR5) 14

3.3.6 Language-Model Correction and Suggestions (FR6) 14
3.3.7 Unicode Insertion (FR7) 14
3.3.8 Text Correction and Editing (FR8) 14
3.3.9 Keyboard Management (FR9) 14
3.3.10 Performance Evaluation (FR10) 14
3.4 Non-Functional Requirements 14
3.4.1 Recognition Accuracy 15
3.4.2 Speed 15
3.4.3 Usability 15
3.4.4 Privacy 15
3.4.5 Size and Resource Use 15
3.4.6 Reliability and Robustness 15
3.4.7 Portability 16

CHAPTER 4: SYSTEM MODELING AND DESIGN 17
4.1 System Design and Analysis 18
4.2 Use Case Diagrams 18
4.3 Full Dress Use Case/Detailed Use Case 18
4.3.1 Write Urdu Handwriting – Full Dress Use Case 18
4.4 Activity Diagram 19
4.5 Data Flow Diagram 19
4.6 System Sequence Diagram 19
4.7 Sequence Diagram 20
4.8 Design Class Diagram 20
4.9 Architectural Diagrams 23
4.9.1 Interface Design 23
4.9.2 Component Level Design 23
4.9.3 Deployment 24

CHAPTER 5: SYSTEM TESTING AND VALIDATION 27
5.1 System Testing 28
5.2 Testing Techniques 28
5.2.1 Unit Testing 28
5.2.2 Integration Testing 29
5.2.3 Black Box Testing 29
5.2.4 Performance Testing 29
5.3 Test Cases 29
5.3.1 Test Case 1: Stroke Capture 30
5.3.2 Test Case 2: Word Recognition 30
5.3.3 Test Case 3: Candidate Selection and Unicode Insertion 31
5.3.4 Test Case 4: Response Latency 32
5.3.5 Test Case 5: Offline Operation and Privacy 33
5.4 Non-Functional Requirements 34

5.4.1 Accuracy Results 34
5.4.2 Speed Results 34
5.4.3 Usability Results 34
5.4.4 Privacy Results 35
5.4.5 Size and Resource Use Results 35
5.4.6 Reliability and Portability Results 35

CHAPTER 6: CONCLUSION 36
6.1 Conclusion 37
6.2 Limitations and Future Work 37

APPENDIX – I 38

REFERENCES 40

LIST OF FIGURES

Figure	Caption	Page No.
4.1 :	Main Use Case Diagram	20
4.2 :	Activity Diagram	21
4.3 :	DFD Level 0	21
4.4 :	DFD Level 1	22
4.5 :	System Sequence Diagram	22
4.6 :	Sequence Diagram	22
4.7 :	Class Diagram	23
4.8 :	Layered Architecture Diagram	24
4.9 :	Interface Design Diagram	25
4.10 :	Component Level Design	25
4.11 :	Deployment Diagram	26

LIST OF TABLES

Table	Caption	Page No.
1.1 :	Hardware requirements	4
1.2 :	Software requirements	5
1.3 :	Dataset totals	5
2.1 :	Summary of Reviewed Literature	8
2.2 :	Summary of Existing Systems	9
4.1 :	Full Dress Use Case for Write Urdu Handwriting	18
5.1 :	Test Case 1: Stroke Capture	30
5.2 :	Test Case 2: Word Recognition	30
5.3 :	Test Case 3: Candidate Selection and Unicode Insertion	31
5.4 :	Test Case 4: Response Latency	32
5.5 :	Test Case 5: Offline Operation and Privacy	33
5.6 :	Non-Functional Requirements, Measures and Results	34

LIST OF ACRONYMS

BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
CER	Character Error Rate
CNN	Convolutional Neural Network
CTC	Connectionist Temporal Classification
DFD	Data Flow Diagram
GAN	Generative Adversarial Network
GNN	Graph Neural Network
HWR	Handwriting Recognition
IME	Input Method Editor
NPU	Neural Processing Unit
ONNX	Open Neural Network Exchange
SDG	Sustainable Development Goal
SSD	System Sequence Diagram
SVM	Support Vector Machine
TFLite	TensorFlow Lite
WER	Word Error Rate

Chapter 1

INTRODUCTION

Rapid digitization has made smartphones a central piece in our life. It changes how we communicate, socialize and work. For Latin-script based languages, text entry on smartphones is not a problem. QWERTY keyboards and glide typing allow easy text input. Even natural handwriting support for many languages is now built-in in modern keyboards. There is no such support for Urdu which is spoken by more than 230 million people around the world and the national language of Pakistan along with being a recognized scheduled language in India.

Urdu is written in cursive and context sensitive Perso-Arabic script. One Urdu letter can take isolated, initial, medial or terminal shape depending on the context. These different forms of letters form 108 unique half-form letteres that must be recognized by the recognizer. Some academic progress has been made on this problem. A preprocessing pipeline which used preclassification and wavelet features by Safdar et et al. [1] successfully achieved 96.1% accuracy on half-form characters and some studies achieved between 85% to 93% accuracy on isolated characters[3, 6, 7, 8]. But these systems are limited to isolated characters. Commercial keyboards are somewhat better but support for Nastaliq style is limited. There is a gap for a mobile keyboard that can be used to recognized Urdu handwriting.

This report presents *Urdu-Ink Mobile*, a mobile and tablet keyboard that can recognize Urdu handwriting in real-time. Users write Urdu in a natural way, the keyboard recognizes the words and converts them into editable Unicode text usable in any application. First, It uses a Neural Network to recongize the words. Second, it introduces a GAN network [10]trained on the collected dataset to generate synthetic sampeles that can increase the total number of samples in the dataset. Third, this system runs completely on-device requiring no internet connection.

1.1 Project Domain

This project falls in the domain of artifficial intelligence, specifically deep learning and pattern recognition for online handwriting recognition. Online handwriting recognition differs from offline recognition as it used a realtime stream of stroke coordinates instead of scanned images. The language processing and recognitin part involves neural networks to recognize the words and the mobile deployment part includes packaging the trained model into a keyboard that runs withing the memory limits of smartphone devices. The problem that is solved in this project is the conversion of continuous Urdu handwriting into its corresponding Unicode text in real-time.

1.2 Problem Identification

Many contributions have been made in the field of Urdu character recognition. The most recent and some of the complete contribution has been made by by Safdar et al. [1], which achieved a 96.1% accuracy in recognizing 108 half-form characters using preclassification and wavelet feature. But these contributions are limited to isolated and short characters. Mobile input using fingers is also a problem due to small screen size and quick writing.

Commercial keyboards do not solve this prolem completely with their limited and basic suppport for Nastalic style Urdu handwriting. The main problem solved by our project is the absence of a word level, real-time and on-device Urdu handwriting recognizer keyboard.

1.2.1 Proposed Solution

Our proposed solution is *Urdu-Ink Mobile*, a system level mobile keyboard that records handwriting data in XY coordinates, time and pressure values and then passes that data through a pre-processing pipeline. The proprocessing pipeline applies jiter removal, down-sampling, interpolation/extrapolation and normalization to the raw handwriting data. A GAN network that is trained on the dataset is used to generate synthetic samples to increase the total number of samples. The preprocessed data is fed into a Neural Network that recognizes the handwritten words. The recognized words are then converted into Unicode text that can be used in any app.

1.2.2 Objectives

The objectives of the project are as follows.

1. To develop a multi-phase word level Urdu handwriting recognition pipeline for smartphones that builds upon the character recognition work of Safdar et al. [1]
2. To train a GAN network that generates synthetic handwriting samples to increase the total number of samples in the dataset
3. To collect a coordintes based Urdu handwriting dataset from native Urdu writers.
4. To protect user privacy and reduce latency by running all inference locally using TensorFlow lite runtime.
5. To test and evaluate the trained models on real mobile devices.

1.2.3 Scope of the Project

The main target of this system is to recognize handwritted Urdu words written in Nastaliq/Naskh style. The system is developed to run on Android 10 and later. The users write on the canvas inside the keyboard app, the trained model recognizes the word and converts it into editable Unicode text to be used in any app such as messaging or email. Recognizing other language words or decorative calligraphic styles in not included in the scope of our project. Educators, journalists, students and other general users are expected users of this system in routine tasks and communications.

1.3 Effectiveness / Usefulness of the System

Our system is different from existing systems as it brings word-level recognition and complete local inference. Our system can be used in any app because it is desinged as a keyboard that can give system-input. Native Urdu speakers who prefer to use Urdu instead of English

can use this system to conveniently write in their native language. It can be used by students for their assignments and note-taking and by geneal users for filling forms and performing other handwriting related tasks. Keeping the security and privacy in mind, the system does not send any handwriting data to the internet and all inference is performed on-device.

1.4 Resource Requirement

General consumer hardware is required for model training, GAN network training and on-device inference. Deep learning frameworks are required for model training. A dataset of coordinates based Urdu handwriting is required for training the recognizer. These requirements are described in more detail in the following subsections.

1.4.1 Hardware Requirement

Consumer grade hardware is required for model and GAN network training. The deployed keyboard can conveniently be run on Android 10+ smartphones. Table 1.1 lists the minimum specification of the target device, which was finalized in the project proposal. It also list the workstation required for training.

Table 1.1: Hardware requirements

Component	**Target device (minimum)**	**Training workstation**
Device	Android/iOS smartphone or tablet	HP/Mechrevo
Processor	Quad-core 1.8 GHz	Ryzen 5 7430U/ Intel I3 13100U
Memory (RAM)	3 GB	8 GB
Storage	500 MB	128
Accelerator	Mobile GPU/NPU recommended	Integrated GPU

1.4.2 Software Requirement

This system uses modern frameworks for training the Neural Network. Python 3.11 is used for data processing and model training, Java is used for Android keyboard development, and C++ for optimizing the model so that it runs smoothly on smartphones. Networks were trained using PyTorch 2.x, Keras for the GAN experiments, and TensorFlow Lite for optimized local inference.Table 1.2 provides a summary of all the software tools.

Table 1.2: Software requirements

Category	**Tools / Languages**
Programming languages	Python 3.11, Java, C++
Deep learning frameworks	PyTorch 2.x, TensorFlow Lite, Keras
Mobile inference runtimes	ONNX Runtime Mobile, CoreML, TensorFlow Lite
Keyboard user interface	Java
Target operating systems	Android 10+

1.4.3 Data Requirement

As the recognizer is a machine learning, it requires a dataset of relevant coordinates based Urdu handwriting samples. Each word is stored in a .bin file starting with a four-byte magic header, followed by the canvas width and height represented as 64-bit floating-point values and then the number of strokes as an unsigned 32-bit integer. Number of points are stored for every stroke. For each point, the values of x,y coordinates and pressure are stored as 64-bit floating points. An unsigned 32-bit integer is used to store the time offset. All these values use little endian order. Many writers contributes samples to the dataset. The dataset is split into training, validation and test sets, with separate writers for training and testing data samples. The dataset is augmented using a GAN network that was trained on the same samples.Total number of writers and samples in the dataset is listed in Table 1.3.

Table 1.3: Dataset totals

Measure	**Total**
Writers	12
Words	996
Samples	1,619

1.5 Report Organization

This report has been organized into six chapters. Chapter 1 introduces our project, its domain, problem, objectives, scope and resource requirements. Chapter 2 includes literature and review of existing systems. It identifies the research gap and defines the boundary of the proposed solution. Chapter 3 lists the modules, functional requirements and non-functional requirements of the system. Chapter 4 the system models and design are covered in this chapter, including the use case, activity, data flow, sequence, class and architectural diagrams. Chapter 5 goes over the testing techniques, test cases and the validation of the non-functional requirements. Chapter 6 includes the conclusion of the report and discusses limitations and future work. An appendix and the list of references is added at the end of the report.

Chapter 2

BACKGROUND AND EXISTING SYSTEMS

The main problem here is the recognition of Urdu handwriting written on a smartphone. The context-sensitive and cursive nature o f Urdu makes this problem even more difficult. Existing solutions only partially solve this problem.

This organization of this chapter is as follows. Section 2.1 contains literature review, Section 2.2 goes over relates systems and applications, Section 2.3 identifies and discusses the problems in existing systeme, and Section 2.4 defines the boundary of the this solution.

2.1 Related Literature Review

Six papers that relate to this project, either through the technique they use or through the problem they address, are discussed in this section.

Safdar et al. [1] propose the methodology for half-form recognition. Their method successfully recognizes 108 half-form characters with 96.1% accuracy. Using this method, they first classify the input character into a smaller group with the help of simple stroke features such as number of strokes and position of diacritics. A support vector machine is used to classify the db2 wavelet features. This preclassification stage helps reduce confusion between similar shpapes and keep the final classifier small.This projects limitations include no word handling segmentation and that this project works only on half-form level.

Razzak et al. [3] use a different approach to recognize Urdu ligatures written in Nastaliq style. They used fuzzy logic and hidden Markov models to recognize 1,800 ligatures with a reported accuracy of 87.6%. This approach is an important step towards complete word recognition as ligatures are the natural units of Urdu words. Limits of this technique include that it depends on handcrafted features.

Hussain and Khan [6] recognized 15 Urdu ligatures using a spatial-temporal neural network with a reported accuracy of 85%. Their work exploits the temporal order of pen movements, a central property of online handwriting. However, the method gives limited evidence of how this method would behave on the full Urdu words or natural writing because it was run on very limited set of ligatures.

Malik and Khan [7] used a holistic approach to achieve 93% accuracy on 39 isolated Urdu characters. This approach of avoiding segmentation by treating eaach character as a single pattern works only for characters and can not be applied to full, connected cursive words because the boundaries between characters are not clear. This approach is therefore limited to isolated characters only.

Nine geometric characters were used by Shahzad et al. [8] to recognize 38 isolated Urdu characters with an accuracy of 92.8%. The accuracy dropped sharply, to about 31%, when tested with non-native writers, as reported by the study. This finding shows that how strongly the recognizers depend on writer styles.

Naz et al. [2] used multi-dimensional long short-term memory (MDLSTM) networks to recognize offline Nastaliq text with 88% accuracy. This work shows that recurrent networks can be used to model the long-range dependencies of cursive Urdu without needing segnmentation. This approach is not suitable for a keyboard becuase it uses images of text instead of pen-trajectory.

Other than Urdu specific studies, this project uses broader deep learning techniques that have been reviewed.Goodfellow et al. [9] introduced generative adversarial networks, which contains generative and adversarial networks that contains two networks (Generative and adversarial). The generative network generates synthetic samples and the discriminator(the adversarial network) tries to distinguish the generated samples from real samples. This forces the generator to improve its sample to a point where they are difficult to distinguish from the real samples.

Table 2.1 summarizes the Urdu-specific papers that were reviewed.

Table 2.1: Summary of Reviewed Literature

Year	**Authors**	**Contribution**	**Techniques**	**Limitations / Remarks**
2022	Safdar et al.	108 half-forms, 96.1% accuracy	Pre-classification, db2 wavelet, SVM	Tablet only; no segmentation; no word level
2010	Razzak et al.	1,800 ligatures, 87.6% (Nastaliq)	Fuzzy logic, HMM	Limited to fixed ligatures
–	Hussain & Khan	15 ligatures, 85%	Spatial–temporal neural network	Tiny dataset
–	Malik & Khan	39 isolated characters, 93%	Holistic recognition	Isolated characters only
–	Shahzad et al.	38 isolated characters, 92.8%	Nine geometric features	Falls to 31% for non-native writers
2014	Naz et al.	Offline Nastaliq text, about 88%	MDLSTM	Offline; no interactivity

2.2 Related Systems/Applications

Several commercial and open-source keyboards were reviewd to understand what is available for Urdu users.

Gboard, developed by Google, supports handwriting input for several languages and works system-wide on smartphones. The support for Urdu is basic and mainly focuses on Naskh-style cursive.

SwiftKey, developed by Microsoft, is best known for word suggestions and is available system-wide. It's support for Urdu handwriting is limited and it best works for users who want to type.

The default keyboard on iOS has no handwriting support for Urdu as it only supports English and Chinese for handwriting. Users who want to write Urdu have to rely on third party apps.

Table 2.2 compares these systems.

Table 2.2: Summary of Existing Systems

Year	System	Contribution	Tools / Technologies	Limitations	Applications
Current	Gboard (Google)	Multi-language handwriting recognition	Neural recognizers, system keyboard	Basic Naskh support; weak on Nastaliq	Messaging, general text
Current	SwiftKey (Microsoft)	Predictive text	Language models, system keyboard	Limited handwriting support	Messaging, general text
Current	Apple iOS Keyboard	English and Chinese handwriting	System keyboard	No Urdu handwriting	Messaging, general text

2.3 Identified Problem from Existing Work

The limitations of the available systems are clear as we have seen in the literature review and evaluation of existing systems. From a technology point of view, academic systems recognize isolated, half-form characters or ligatures using classic classifiers like vector mahines and hidden Markov models and they were tested on tablets instead of small, noisy screens of smartphones. The only approach that works reliably and properly, based on MDLSTM [2], uses scanned images and offers no interactivity. From a functional point of view, the reviewed projects do not recognize complete words and provide no system-wide support for input. From a non-functional point of view, the commercial keyboards provide limited support for Urdu handwriting an none of the academic systems provide privacy protection on smartphone devices.

2.4 Selected Boundary for Proposed Solution

Our proposed systems provides following functionalities. A system-wide keyboard canvas captures XY coordinates, time and pressure values of handwriting strokes on the smartphone screen. The captured data is then passes through a pre-processing pipeline that removes jitter, downnsamples and interpolate/extrapolate the stroke. These techniques ensure that the data that will be passed to the recognizer is clean and has a fixed number of points for every word. The stroke is normalized before passing it to the Neural Network recognizer for recognition. A GAN will also be used to increase the number of total samples in the dataset. All inference is run on-device using TensorFlow Lite so that the keyboard works offline and protects user privacy. There are several features that are not implemented in this project. The recognizer recognizes only Urdu and no other language. The system also does not recognize decorative or calligraphic writing as it would require more training data to achieve that. Othe accessibility features such as translation are outside of the scope of this project. Deployment on every Linux-based operating system are left as future work.

Chapter 3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

This chapter explains the requirements and specifications of Urdu-Ink Mobile: what the system does, how its different parts work together, and how well it needs to perform. It describes how the system behaves(functional requirements) and how reliable and accurate it must be(non-functional requirements).All requirements directly follow the scope and objectives given in Chapter 1.

The chapter is organized as follows. Section 3.1 gives the system specification, Section 3.2 describes the system modules, Section 3.3 lists the functional requirements, and Section 3.4 describes the non-functional requirements together with the way each one is measured.

3.1 System Specification

The system specification defines the technical boundaries for the processing system. The model processes cursive, context-sensitive Nastaliq/Naskh Urdu handwriting supporting more than 108 half-form shapes, full ligatures, words and complete sentences on smartphones and tablets running Android 13, iOS 16 or later with a possible port to Linux-based mobile OS. It works as real-time, system-level keyboard and is available in every application. The precessing pipeline runs on standard hardware (such as quad-core 1.8 GHz processor, 3 GB of RAM, and 500 MB of Storage, as shown in Table 1.1). The following sections define the functional modules, features and quality requirements that buildup on this setup.

3.2 System Modules

A module is a separate part of a software system that performs a specific task. Splitting the system into modules makes it easier to develop, train, test and update each part independently. It also allows complex models to be optimized for mobile deployment.

Urdu-Ink Mobile is divided into eleven modules. These modules follow the handwriting recognition process, starting from capturing handwriting on the touchscreen and ending with the insertion of recognized Unicode text.

3.2.1 Data Acquisition Module

This module is responsible for capturing handwritten input. A specialized Input Method Editer (IME) interface recieves touch events directly from the devices's touchscreen and a stroke logger records the coordinate values, pressure and time of every point in each stroke, including those written close together in time. It records and saves handwriting samples in the binary stroke format described in Section 1.4.3 in data collection part.

3.2.2 Urdu Handwriting Corpus Module

Handwriting data is stored and managed by this module to train and evaluate the models. The dataset contains handwriting samples collected from different native Urdu writers. It covers half-forms, ligatures, and complete words written in continuous cursive handwriting.

The writers included in the testing set are kept separate from those used for training. This helps measure how accurately the system recognies handwriting from writers it has not seen before.

3.2.3 Preprocessing Engine

The preprocessing engine prepares the captured handwriting before it is passed to the neural networks in order to improve the quality and consistency of input data. Resampling places points at uniform distances from each other. Smoothing reduces unwanted roughness caused by finger movement. Normalization adjusts the size and pressure values so that the system is not affected by differences in writing size and pressure. Denoising removes accidental touches and electronic interference. These steps help provide consistent input to the remaining modules and improve the overall recognition process.

3.2.4 GAN-Based Stroke Generation Module

This module uses Generative Adversarial Network (GAN) to generate Urdu handwriting stroke sequences. GAN consists of a generator and a discriminator. The generator learns to produce ealistic stroke sequences, while the discriminator evaluates whether the generated sequences resemble genuine Urdu handwriting strokes. The generated strokes sequences can help improve the variety of training data.

3.2.5 Ligature Segmentation Module

Urdu handwriting is written in connected cursive forms called ligatures so the system needs to divide the continuous handwriting into smaller parts for recognition. This module uses a skeleton-based approach to perform segmentation: reducing the writing shape to its basic skeleton first and then identifies natural separation points in the connected handwriting. These points help divide a word into smaller units, such as ligatures or characters to improve the accuracy of recognition module. The approach is based on skeleton-based grapheme segmentation used for cursive Arabic-script writing [4]

3.2.6 Pre-Classifier Module

To save processing time and avoid mistakes, a Graph Neural Network (GNN) pre-classifier checks stroke shapes, stroke counts, and dot positions. It then routes the writing into 1 of 28 smaller shape groups, so the main system only has to pick from a small group instead of all 108 half-forms.

3.2.7 Hybrid Feature Extraction Module

This module converts each segmented ligature into a single numerical vector combining three feature types: db2 wavelet transforms for multi-scale trajectory shapes, structural properties (dots, loops, direction changes), and CNN embeddings for visual patterns. Merging these features allows the classifier to use both handcrafted and learned data.

3.2.8 Recognition Core

The recognition core combines a Transformer encoder for long-range sequence dependencies with a BiLSTM and CTC decoder for unaligned character prediction [12, 5]. Domain-adversarial training [14] makes the learned features writer-independent, replacing earlier SVM and basic RNN approaches to better handle cursive Urdu.

3.2.9 Language Model Layer

To fix visual mistakes, this module uses an Urdu N-gram model and BERT-Urdu rescoring [13] to score word likelihood based on context. It picks the most accurate word among visually similar choices and outputs a ranked list of suggestions to the user.

3.2.10 Real-Time Inference Engine

The inference engine executes the trained models locally on the device using TensorFlow Lite, ONNX, or CoreML formats. The models are quantized to reduce their size and run efficiently on the mobile GPU or NPU. Running everything locally keeps latency low, allows the keyboard to work offline, and ensures that handwriting data remains private.

3.2.11 Evaluation and Analytics Module

This module measures system performance by calculating Character Error Rate (CER), Word Error Rate (WER), and latency against test samples. A developer dashboard displays a confusion matrix to identify common character mistakes and guide dataset or model improvements.

3.3 Functional Requirements/Software Features

Functional requirements state what the system must do. The requirements below are derived from the objectives and scope of the project and are each linked to one or more of the modules in Section 3.2. They are used in Chapter 5 to design the test cases.

3.3.1 Stroke Capture (FR1)

The system shall capture finger strokes on a writing canvas inside the keyboard, recording the position, pressure, and time of every point, and shall support multiple strokes for a single word, including late-written dots and diacrtical marks.

3.3.2 Stroke Preprocessing (FR2)

In order for the recognition stages to receive input of uniform quality regardless of the speed, size or pressure of the writer, the system shall resample, smooth, normalize, and denoise the captured strokes.

3.3.3 GAN-Based Stroke Generation (FR3)

The system shall use a GAN to generate Urdu handwriting stroke sequences. It shall generate the major strokes that form the main structure of a word, followed by the minor strokes that complete the word. The generator and discriminator shall work together during training to improve the quality and realism of the generated stroke sequences.

3.3.4 Segmentation and Pre-classification (FR4)

Using skeleton-based segmentation, the system shall split continuous cursive writing into ligatures, and shall route each ligature to one of the 28 subsets by using graph neural network.

3.3.5 Handwriting Recognition (FR5)

By using the hybrid feature extractor and the Transformeer with BiLSTM-CTC classifier, the system shall recognize half-forms, ligatures, and words of Urdu cursive handwriting and shall output a character sequence fore each written word.

3.3.6 Language-Model Correction and Suggestions (FR6)

Recognition errors shall be corrected using N-gram model and BERT-Urdu by the system, alongwith display of a ranked list of candidate words for user choice and selection.

3.3.7 Unicode Insertion (FR7)

The system shall insert the accepted word into the text field of the active application as Unicode text allowing result editing and searching by that application.

3.3.8 Text Correction and Editing (FR8)

Users should be allowed to delete the last character or word, to clear the canvas, to rewrite a word and to insert a space or a new line withoout leaving the keyboard.

3.3.9 Keyboard Management (FR9)

The system shall be installable as a system keyboard allowing the users to switch between the Urdu-Ink Mobile and other installed keyboards.

3.3.10 Performance Evaluation (FR10)

An evaluation mode shall be provided by the system in which CER, WER and latency are calculated for a labelled test set and in which a confusion matrix is displayed for developers.

3.4 Non-Functional Requirements

Non-functional requirements describe the quality and performance of the system rather than the specific tasks it performs. Each requirement with its purpose and a way to measure so

it can be verified in Chapter 5. The numerical values represent the performance targets set for the project.

3.4.1 Recognition Accuracy

Frequent recognition mistakes would force users to correct their text repeatedly making the keyboard difficult to use. Recognition accuracy, the most important quality attribute, is measured using Character Error Rate (CER) and Word Error Rate (WER) on a test dataset containing handwriting from writers who were not included in the training data. The target is a CER of 8% or less and a WER of atmost 20% for unseen writers, and a half-form accuracy that is not lower than the 96% reported in the base study [1] using comparable data.

3.4.2 Speed

The system must display recognition results quickly after pen lift to ensure a natural writing experience. Speed is measured as the time between the end of a word and the display of the candidate on the screen. Average response time must not exceed 300 ms, with 95% value at most 600 ms.

3.4.3 Usability

Usability is how easily users can learn and operate the keyboard. It is important as the target users include people who may not be comfortable using English keyboards. It is measured by observing new users and collecting feedback through System Usability Scale (SUS) questionnaire. The system is usable if a new user can write and insert a first word within two attempts after receiving a one-minute demonstration. The avergae SUS score should be atleast 70.

3.4.4 Privacy

Handwriting may contain personal or private information, protecting user's content is an important requirement. To protect user privacy, all handwriting strokes and data must be processed locally on the device. Privacy will be tested by checking network activity while the keyboard is being used in offline-core mode. The system must operate without transmitting any data, stroke information, or processed text over the internet.

3.4.5 Size and Resource Use

The keyboard system must use minimal system resources so that it can operate on mobile devices with limited resources. Total storage usage (including all data models) must stay under 500 MB, and peak memory usage during processing must not exceed 3 GB of RAM

3.4.6 Reliability and Robustness

Reliability measures how many failures occur during continuous operation, and robustness as the required time to recover from failure. The system must operate continuously with-

ous crashing, targetting at least 1,000 consecutive recognized words without failure. If an unexpected error occurs, the system should recover in under 2 seconds without losing any previously inserted text.

3.4.7 Portability

The system is intended to support major mobile platforms. Portability will be measured by checking the number of tarrrget operating systems on which the keyboard can be installed nad pass its functional tests, alongwith the amount of recognition code shared between platforms. The requirement is to pass the the functional tests on Android 13, iOS 16 or later for both, with the same trained models used for both.

Chapter 4

SYSTEM MODELING AND DESIGN

The requirements of Chapter 3 are converted into models that developers and reviewers can review before and during the implementation. Urdu-Ink Mobile is an interactive system with a user who writes on the screen, a pipeline of processing stages, and a host application that receives the results. This chapter shows the functions that are available to users using flow activities, flow of data, the interaction between user and the system and the architecture of the whole system.

The chapter is organized as follows. Section 4.1 gives an overview of the models used, Section 4.2 presents the use case diagram, Section 4.3 gives the full dress use cases, Section 4.4 presents the activity diagram, Section 4.5 the data flow diagrams, Section 4.6 the system sequence diagram, Section 4.7 the sequence diagram, Section 4.8 the design class diagram, and Section 4.9 the architectural diagrams.

4.1 System Design and Analysis

Analysis models explain the required functions of the system while design models present the design of the system. The following diagrams are used in this project. Use case diagrams and full dress use cases describe the functions from user point of view.

4.2 Use Case Diagrams

A use case diagram is used to show the functions of the system that requie user interaction. It also helps to view the system from users perspective.Figure 4.1 shows the main use case diagram of Urdu-Ink Mobile. User is the primary actor here who writes words on the canvas and configures the keyboard.

4.3 Full Dress Use Case/Detailed Use Case

Full dress use cases are used to describe the main scenario and the alternatives of a use case in detail. They are only written for the use cases that are complex and require explanation. Write Urdu Handwriting use case is detailed using a full dress use case diagram becuase it contains most of the functional requirements of the project.

4.3.1 Write Urdu Handwriting – Full Dress Use Case

Table 4.1 explains the full dress use case in which user writes a word on the canvas and the system recognizes the word.

Table 4.1: Full Dress Use Case for Write Urdu Handwriting

Use Case Name	Write Urdu Handwriting
Scope	Urdu-Ink Mobile keyboard
Level	User goal
Primary Actor	User

Stakeholders and Interests	User: wants fast and correct conversion of handwriting into text. Application owner: wants the host application to receive valid Unicode.
Pre-conditions	The keyboard is installed and selected as the active input method; a text field of a host application has focus; the models are loaded.
Main Success Scenario	1. The user writes a word in continuous cursive on the canvas. 2. The system captures the strokes. 3. The system removes gitter, downsamples, interpolates/extrpolates and normalizes the strokes. 4. The system passes the preprocessed data to the recognizer. 5. The system inserts the recognized word into the target textbox.
Extensions	5a. Word not in the vocabulary: the system shows the raw recognition result.
Special Requirements	All processing is local (Section 3.4.4).

4.4 Activity Diagram

An activity diagram is a flowchart that is used to show the flow from one activity to another. It is used here to give an overview of the system's function regarding recognition. Figure 4.2 shows the activity flow of a session. The user writes a word, which is preprocessed and then passed to the recognizer neural network. The recognizer recognizes the word and inserts the reconized word's Unicode the target textbox.

4.5 Data Flow Diagram

A data flow diagram is used to show the flow of data in an application. It shows the inputs and outputs of each entity and process without the control flow or decisions. A data flow diagram is used for this project because the system recieves input from users in the form of strokes, preprocesses it and then returns the Unicode of recognized words. Figure 4.3 shows the level 0 diagram, which shows the system as a single process which exchanges data with three entities. The user supplies the data in the form of handwriting strokes and receives the recognized word as output. The host application receives Unicode text and the developer provides test samples and receives evaluation results.

Figure 4.4 decomposes the system into 4 processes. Strokes written by the user are captured(1.0), strokes are preprocessed(2.0), recognized by the neural network and sends the Unicode to host application(3.0) and an evaluation process that compares the recognized words against actual labels(4.0). The recognizer is stoed in the trained model store(D1) and the evaluation process reads the labelled test samples(D2).

4.6 System Sequence Diagram

The input and output events that occur between the actor and the system treated as a blackbox are illustrated using a system sequence diagram. It is a form of use case diagram. A system sequence diagram for the Write Urdu Handwriting of section 4.3, is given here so that the

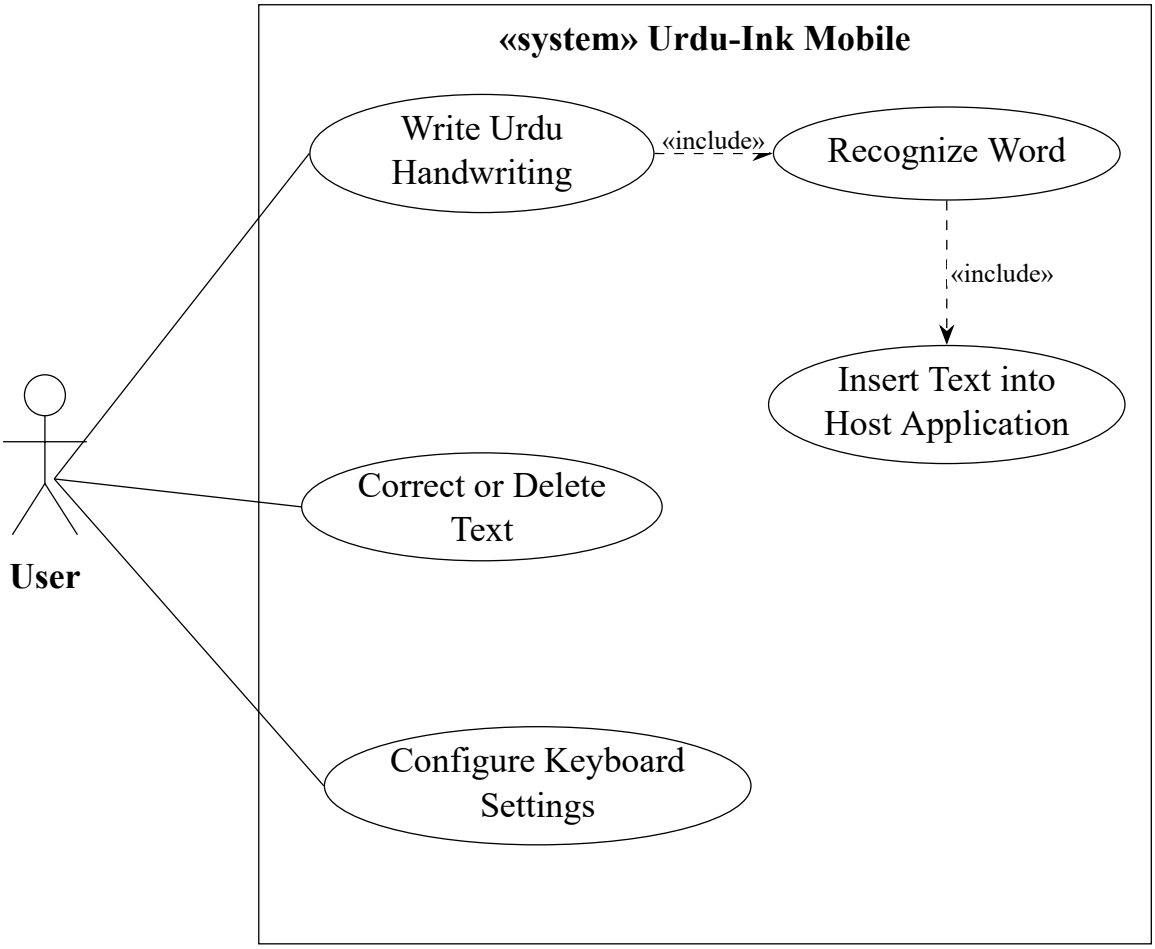


Figure 4.1: Main Use Case Diagram

external events that the keyboard has to handle are clear. Figure 4.5 shows that a user writes an Urdu word, sends the strokes to the system after completing writing the word and receives the Unicode text as a result.

4.7 Sequence Diagram

A sequence diagram shows the interaction between objects in the order in which they exchange data and messages. It is used here to show the event of recognition and insertion of one handwritten word.Figure 4.6 shows that the strokes of handwriting are captured using the input method service. The captured strokes are passed to the preprocessing pipeline which prepates the raw strokes for recognition. The cleaned strokes are then sent to the nueral netwrok, which recognizes the word and returns the Unicode text to the host application.

4.8 Design Class Diagram

The desing class diagram shows the main classes of the system, their properties and functions and the relationship between them. UrduInkIME is the main class, which captures the handwriting strokes. The UrduInkSession holds the stroke objectss that will be passed to

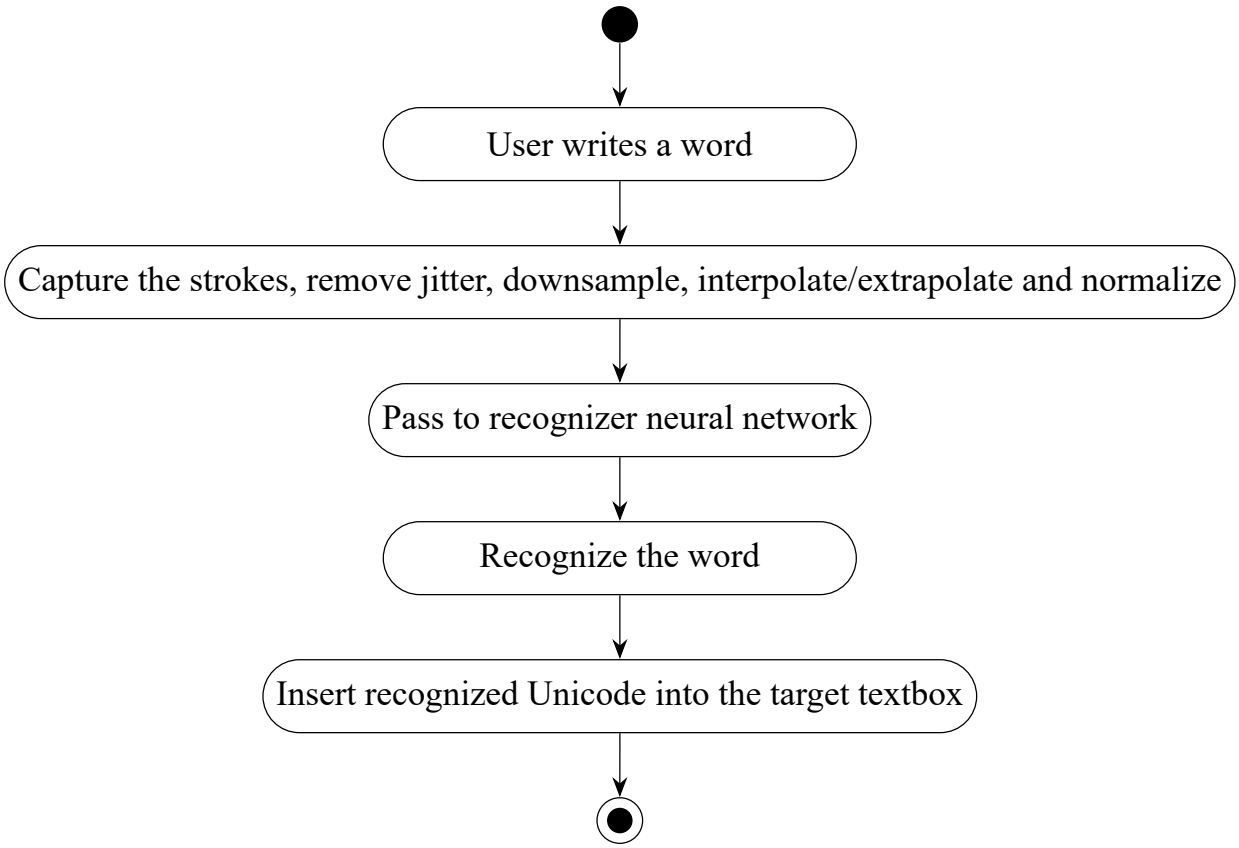


Figure 4.2: Activity Diagram

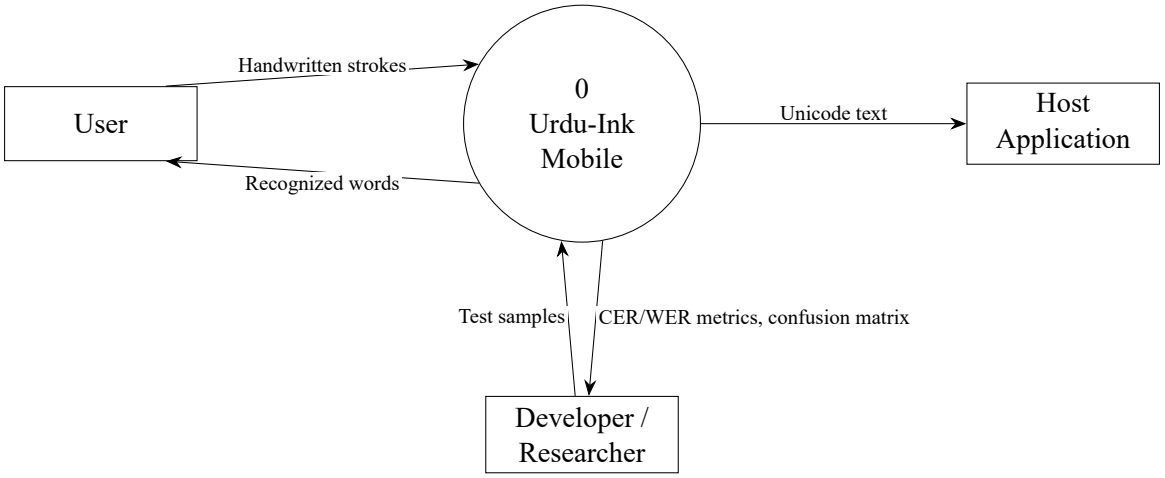


Figure 4.3: DFD Level 0

the preprocessor. The preprocessor class preprocesses the strokes to make them ready for recognition. It then passes clean strokes to the recognizer. The recognizer recognizes the handwriten word. The only class that interacts directly with the operating system and keeps the corresponding code separate from the recognition code. The UrduInkSession holds one or more strokes with their corresponding coordinates, pressure and time offset values. The InferenceEngine hides the differences between the mobile runtimes, so that a model can be exchanged without changes to the other classes. The numbers on the arrows in the pipeline show the order in which the classes are called for one handwritten word.

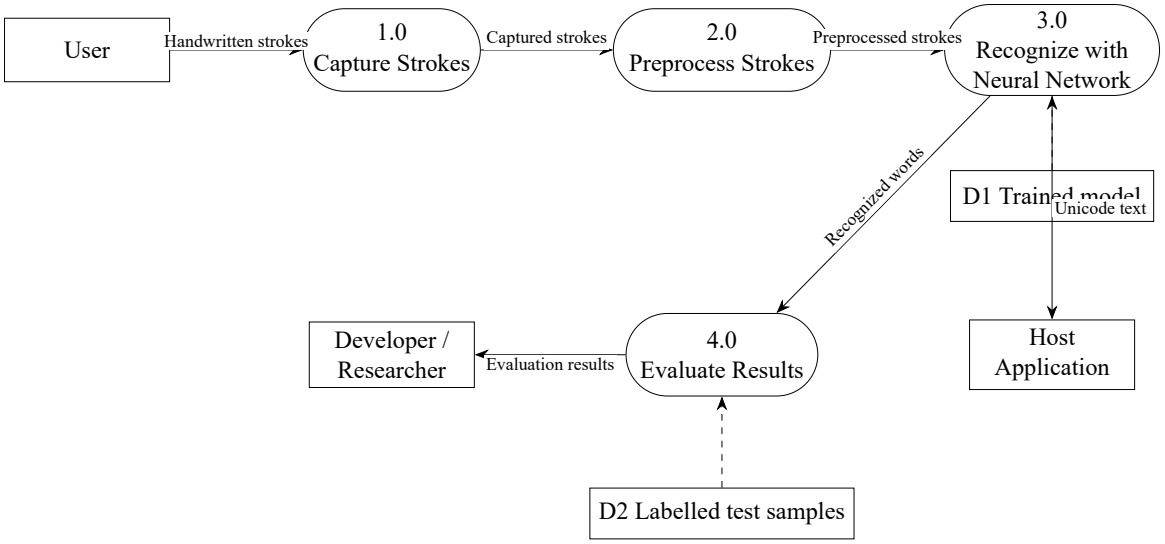


Figure 4.4: DFD Level 1

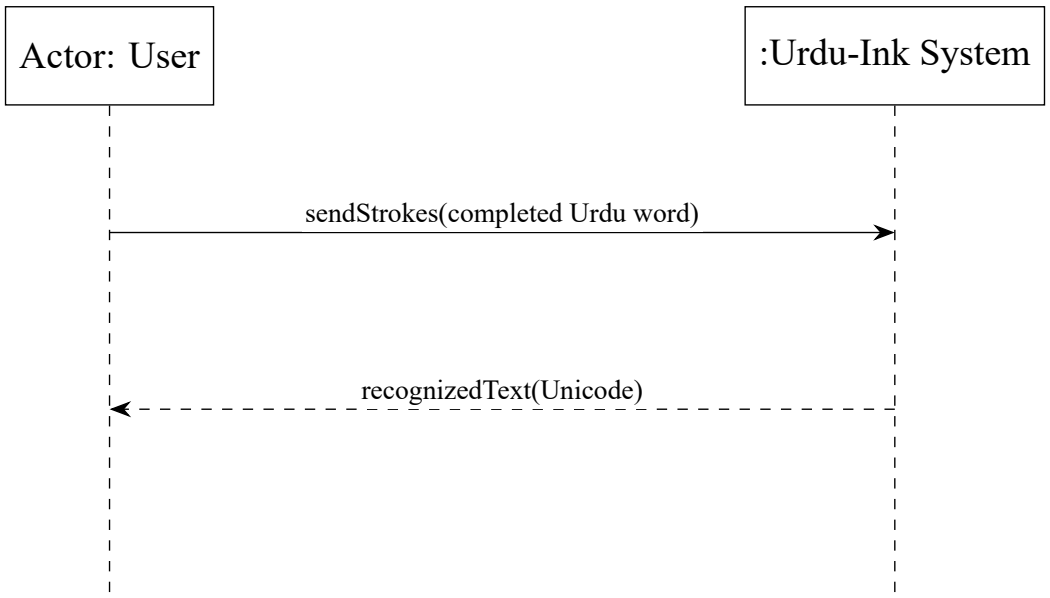


Figure 4.5: System Sequence Diagram

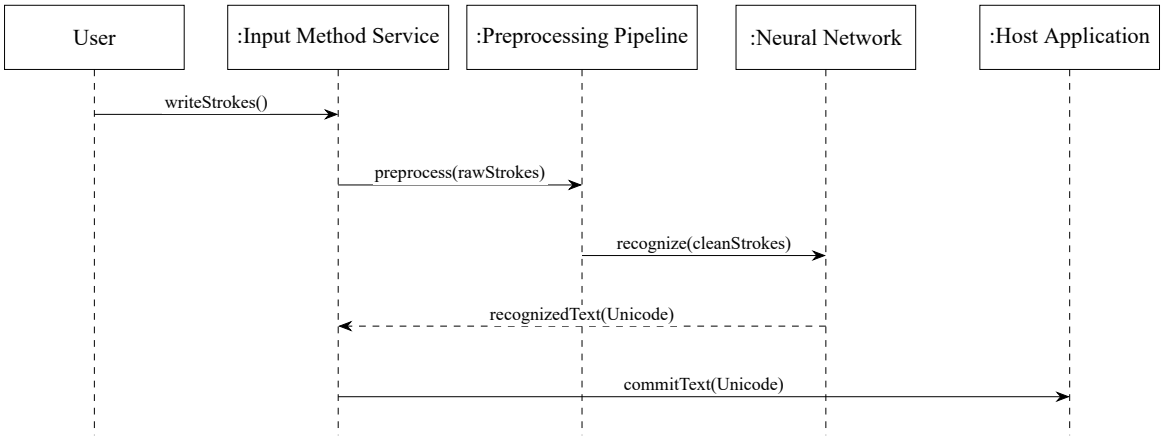


Figure 4.6: Sequence Diagram

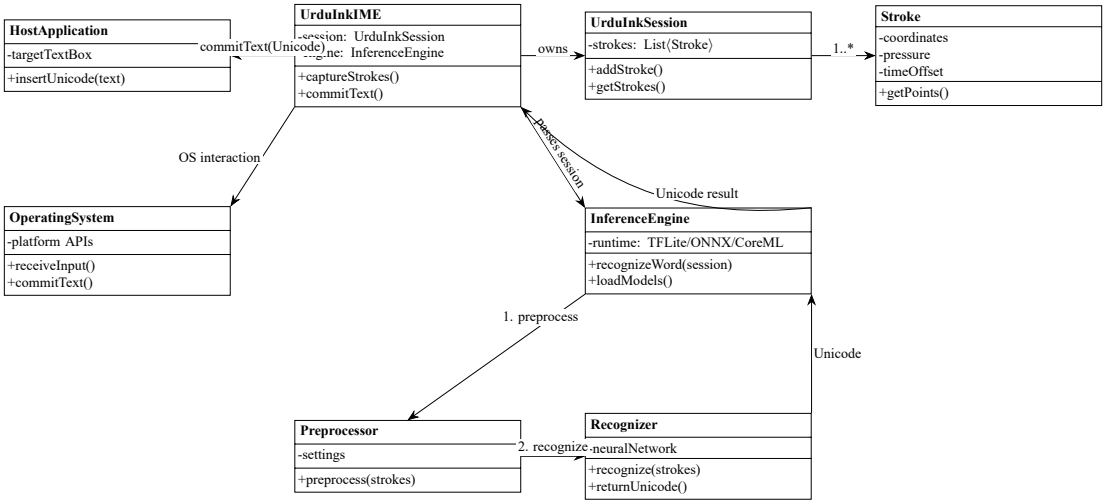


Figure 4.7: Class Diagram

4.9 Architectural Diagrams

An architecture provides a blueprint of a project that gives abstraction to reduce complexity and explain how different components communicate with each other. A layered architecture is followed for UrduInk Mobile becuase every layer can be replaced without changing any other component of the system. For example when a new runtime or recognizere is available, it can be swapped without changing anything else.Figure 4.8 shows the five layers of the system. The presentation layer contains the keyboard user interface. The input layer captures the required stroke data. The preprocessing layer cleans the raw strokes to prepare for recognition. The next layer is recognition layer that recognizes the handwritten word. Then the runtime and platform layer executes the model on the operating system. This arrangemnet of layers also supports testing as each layer can be tested with input from the layer above. This arrangement also allows portability as only the presentation, input and paltform layer contains platform specific code.

4.9.1 Interface Design

The interface desing shows the interface of the keyboard as the user sees it. Figure 4.9 presents the wireframe. The upper part of the layout shows the textbox of th host application. Below the textbox is the writing canvas with a dashed line that helps users write on a straight line. A row below the canvas holds the keys for switching keyboards, inserting a space, deleting and inserting a new line and a button for settings. This row also holds a button to signal the end of the word and send the strokes for recognition. All the buttons can be reached with a single hand and allow the user to enter the words without leaving the keyboard.

4.9.2 Component Level Design

The component diagram shows the software components and the interfaces that they communicate through. In Figure 4.10 the keyboard user interface passes the strokes to the stroke-

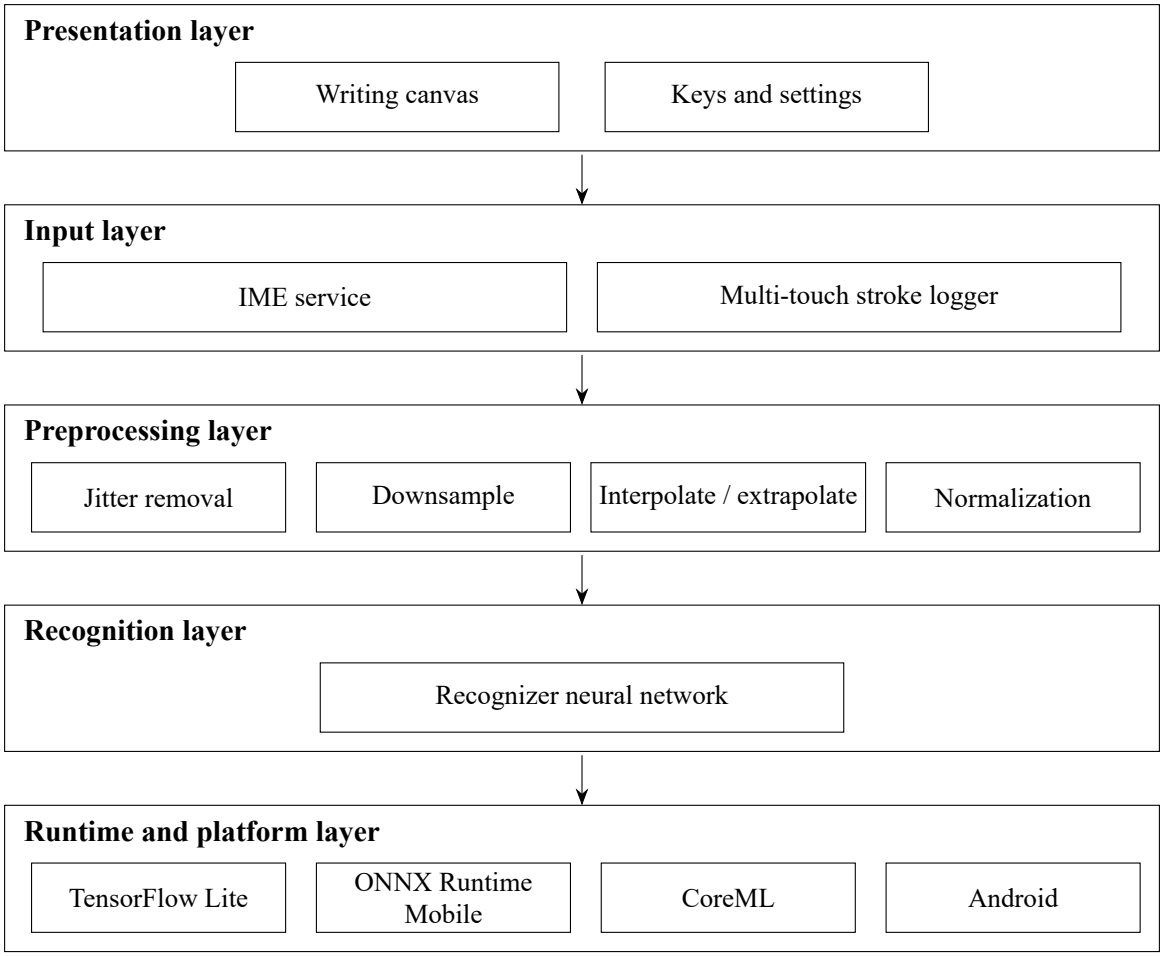


Figure 4.8: Layered Architecture Diagram

input interface to the InkCapture component. The InkCapture component passes the raw strokes to the preprocessing engine that cleans the strokes and prepares them for recognition. The preprocessed and cleaned strokes are then passed to the neural network for recogniton. The recognized word is returned to the keyboard user interface. The neural network component is executed by the Inference Runtime component, which abstract the differece between TensorFlow Lite, ONNX Runtime Mobile and CoreML.

4.9.3 Deployment

The deployment diagram shows the physical devices the model is trained and runs on. Figure 4.11 shows two devices. The model is trained on a consumer laptop where the python training script and the dataset are stored and is then exported in mobile formats. The exported model is bundled with the keyboard application that is installed on user's smartphone where the keyboard application, the model files, the inference runtime and user settings are stored. The keyboard interacts with the target application using the input method services of the operating system and no data leaves the user's device. Keeping the training and deployment of different devices also shows the different requirements of the two phases. Training requires a better GPU and the full training dataset while deployment only requires the quantized, compact model and runtime that fits withing the RAM and storage constraints of the

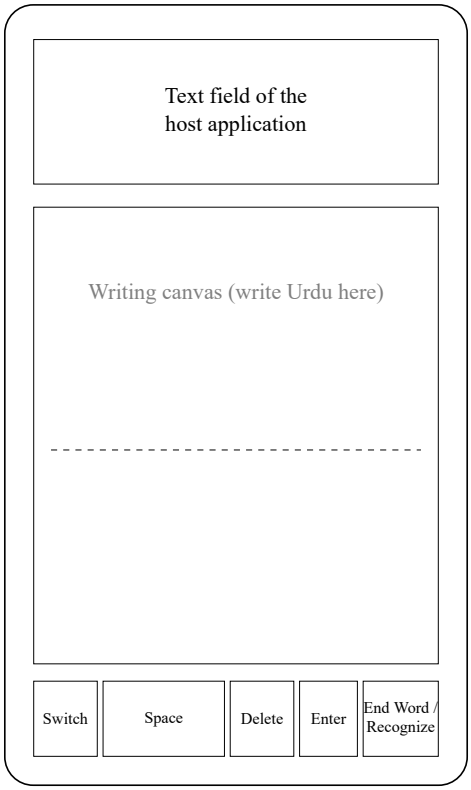


Figure 4.9: Interface Design Diagram

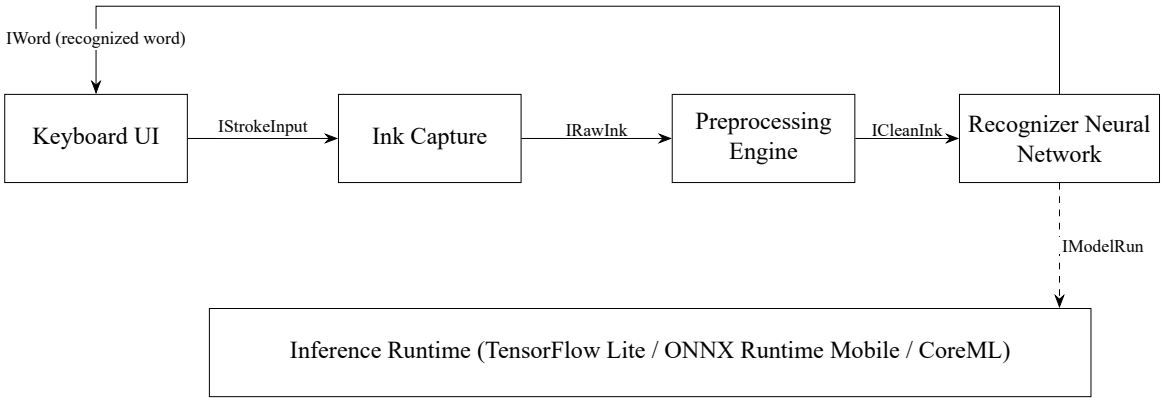


Figure 4.10: Component Level Design

user's smartphone.

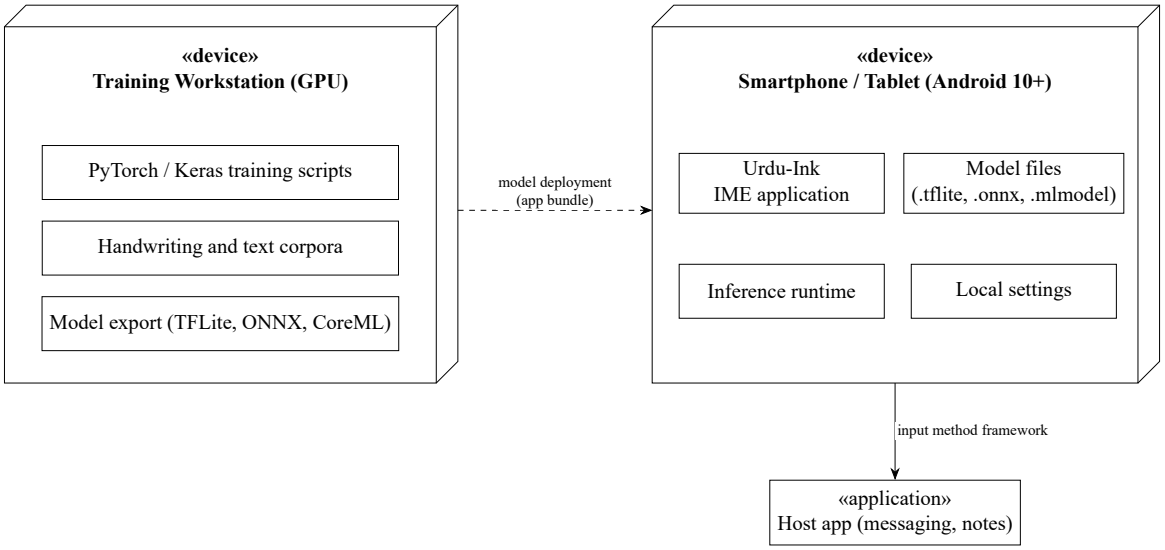


Figure 4.11: Deployment Diagram

Chapter 5

SYSTEM TESTING AND VALIDATION

This chapter describes how Urdu-Ink Mobile was tested and how the results were validated against the objectives and requirements of the earlier chapters. It covers the overall approach to system testing, the testing techniques that were used, the test cases that were designed for the highest-priority requirements, and the verification of the non-functional requirements with measurable criteria.

The chapter is organized as follows. Section 5.1 gives an overview of system testing and the analysis of the results, Section 5.2 describes the testing techniques, Section 5.3 presents the test cases, and Section 5.4 reports on the non-functional requirements.

5.1 System Testing

System testing checks whether the complete, integrated system satisfies its requirements. It is important for a handwriting keyboard because the quality of the product is judged by what the user experiences, which is a result of many interacting stages rather than any single model. A network that scores well in isolation may still produce a poor keyboard if the segmentation is wrong, if the latency is high or if the text is not inserted correctly into other applications. Four techniques were applied: unit testing of individual modules, integration testing of the pipeline, black-box functional testing from the point of view of the user, and performance testing on real devices.

The overall analysis of the test results must be consistent with the objectives of Section 1.2.2. The analysis for the first and second objectives is based on the recognition accuracy of the full pipeline with and without the RefinerGAN, the analysis for the third and fourth objectives on CER and WER for unseen writers with and without language-model correction, the fifth objective on latency and the network-traffic test, and the sixth objective on the complete set of results of this chapter. [Write two or three sentences here summarizing the actual outcome once all tests are completed, referring to Table 5.6 and the test cases in Section 5.3.]

5.2 Testing Techniques

Testing techniques differ in what they look at. White-box techniques examine the internal structure of the code, black-box techniques examine only inputs and outputs, and techniques such as unit and integration testing are applied at different levels of the system. The techniques used in this project are described in the following subsections.

5.2.1 Unit Testing

Unit testing verifies each module separately by running it on prepared inputs and comparing its output with the expected output. It was used because the pipeline has many independent modules and an error in an early module spreads to all later ones. The preprocessing functions were tested with synthetic strokes whose correct resampled and smoothed forms are known, the segmenter with words of known ligature structure, and the data-reader with binary files created by hand to check the byte layout of the corpus format. The models were

tested by checking their output shapes and by evaluating them on held-out data. [Report the number of unit tests and how many passed.]

5.2.2 Integration Testing

Integration testing checks that modules work together through their interfaces. It was applied because several stages must agree on data formats, such as the coordinate scaling between the preprocessor and the RefinerGAN and the sequence format between the recognition core and the language model. The pipeline was integrated step by step: first the preprocessor with the RefinerGAN, then the segmenter and pre-classifier, then the recognition core, and finally the language model and the input method service. After each step a set of recorded handwriting samples was passed through the combined modules and the outputs were compared with the outputs of the separate modules. [Report any interface defects found and how they were fixed.]

5.2.3 Black Box Testing

Black-box testing checks the behavior of the system against the functional requirements without looking at the code. It was used to test the keyboard as users experience it, in real applications. Testers wrote Urdu words on the canvas, selected candidates, deleted text and switched keyboards in messaging and note-taking applications, and checked the resulting text and the behavior of the interface against the expected results in the test case tables of Section 5.3. Testers included writers who were not part of the training data. [Report the number of testers and the number of test scenarios.]

5.2.4 Performance Testing

Performance testing measures speed and resource use. It was applied because the main promise of the product is real-time recognition on ordinary phones. Latency was measured as the time between the end of a word and the display of the candidates, over a large number of words on the reference device and on at least one higher-end device, and memory and storage use were read from the profiling tools of the operating system. [Describe the exact tools used, such as Android Studio Profiler or Xcode Instruments.]

5.3 Test Cases

A test case specifies the input, the conditions and the expected result for a function that is to be tested, so that the test can be repeated and the result judged as pass or fail. Not every function is tested exhaustively. The test cases were chosen according to the priority of the requirements and the risks of the project. Five test cases were designed: stroke capture (FR1), word recognition (FR2 to FR5), candidate selection and Unicode insertion (FR6 and FR7), response latency, and offline operation. Each test case follows the standard template in Tables 5.1 to 5.5.

5.3.1 Test Case 1: Stroke Capture

This test case checks that the keyboard captures every stroke on the canvas, including late-written dots and diacritical marks, as required by FR1. Its results are recorded in Table 5.1.

Table 5.1: Test Case 1: Stroke Capture

GENERAL INFORMATION	
Test Case Number / Stage	TC-01 / Functionality
Test Case Description	Verify that all strokes written on the canvas are captured with position, pressure and time.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	FR1 – Stroke Capture
Set-up and Stop Procedures	Set-up: install the keyboard, enable it in the system settings and focus a text field. Stop: clear the canvas and close the application.
ENVIRONMENTAL NEEDS	
Hardware and Software	Android 13 smartphone with the minimum specification and an iOS 16 device; keyboard build [version].
TEST	
Inputs and Procedural Steps	Write ten words, each with at least two strokes and with dots written after the main body; inspect the logged stroke count and point coordinates for each word.
Expected Results	The logged stroke count equals the number of strokes written for all ten words, point timestamps increase monotonically, and no stroke is missing or duplicated.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.2 Test Case 2: Word Recognition

This test case checks the end-to-end recognition accuracy of continuous cursive words for writers who were not part of the training data, which relates to FR2 to FR5 and to the accuracy requirement. Its results are recorded in Table 5.2.

Table 5.2: Test Case 2: Word Recognition

GENERAL INFORMATION	
Test Case Number / Stage	TC-02 / System

Test Case Description	Verify that written words are recognized with CER and WER within the targets.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	FR2, FR3, FR4, FR5; Recognition accuracy (Section 3.4.1)
Set-up and Stop Procedures	Set-up: load the held-out test set recorded by unseen writers and the trained models. Stop: save the recognition results and the confusion matrix.
ENVIRONMENTAL NEEDS	
Hardware and Software	Reference device with the minimum specification; evaluation mode of the keyboard; Python scripts to compute CER and WER.
TEST	
Inputs and Procedural Steps	Pass the [number] test words through the pipeline and compare the output with the ground-truth text; repeat with the RefinerGAN disabled and with the language model disabled to measure their contribution.
Expected Results	CER is at most 8% and WER is at most 20%; the full pipeline gives lower error rates than the pipeline without the RefinerGAN and the pipeline without the language model.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.3 Test Case 3: Candidate Selection and Unicode Insertion

This test case checks that the user can select a suggested word and that the correct Unicode text arrives in the host application, as required by FR6 and FR7. Its results are recorded in Table 5.3.

Table 5.3: Test Case 3: Candidate Selection and Unicode Insertion

GENERAL INFORMATION	
Test Case Number / Stage	TC-03 / Integration
Test Case Description	Verify that a selected candidate is inserted into the active text field as correct Unicode text.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	

Requirement(s) Tested	FR6 – Language-Model Correction and Suggestions; FR7 – Unicode Insertion
Set-up and Stop Procedures	Set-up: open a messaging application and a notes application and focus a text field. Stop: delete the inserted text.
ENVIRONMENTAL NEEDS	
Hardware and Software	Android and iOS devices; messaging and note-taking applications installed on the device.
TEST	
Inputs and Procedural Steps	Write a word, tap the first, second and third candidate in turn on separate attempts, and read back the text inserted in each application.
Expected Results	The inserted text equals the tapped candidate in both applications; the canvas and candidate bar are cleared after insertion; the text can be copied and searched by the application.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.4 Test Case 4: Response Latency

This test case measures how quickly candidates appear, which is the speed requirement. Its results are recorded in Table 5.4.

Table 5.4: Test Case 4: Response Latency

GENERAL INFORMATION	
Test Case Number / Stage	TC-04 / Performance
Test Case Description	Measure the time from the end of a word to the display of candidates.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	Speed (Section 3.4.2)
Set-up and Stop Procedures	Set-up: enable the timing log and restart the keyboard so that models are freshly loaded. Stop: export the log file.
ENVIRONMENTAL NEEDS	
Hardware and Software	Reference device with quad-core 1.8 GHz processor and 3 GB of RAM; a higher-end device for comparison.
TEST	

Inputs and Procedural Steps	Write 200 words of different lengths at normal writing speed and record the latency of each word from the log.
Expected Results	The average latency is at most 300 ms and the 95th percentile at most 600 ms on the reference device.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.3.5 Test Case 5: Offline Operation and Privacy

This test case checks that recognition works without a network and that no handwriting leaves the device, as required by the privacy requirement. Its results are recorded in Table 5.5.

Table 5.5: Test Case 5: Offline Operation and Privacy

GENERAL INFORMATION	
Test Case Number / Stage	TC-05 / Acceptance
Test Case Description	Verify that recognition works in airplane mode and that no network traffic is generated by it.
Test Date & Tester	[dd/mm/yyyy] [tester name(s)]
Result	[Pass / Fail] Incident number: [if applicable]
INTRODUCTION	
Requirement(s) Tested	Privacy (Section 3.4.4); FR5
Set-up and Stop Procedures	Set-up: switch the device to airplane mode, and in a second run connect it to a network monitor. Stop: restore the network settings.
ENVIRONMENTAL NEEDS	
Hardware and Software	A smartphone with a packet-capture or network-monitoring tool.
TEST	
Inputs and Procedural Steps	With airplane mode on, write twenty words and record the result; with the network on, repeat the writing while monitoring all outgoing connections of the keyboard process.
Expected Results	Recognition succeeds for all words in airplane mode, and the monitor shows no outgoing connection carrying strokes or text from the recognition path.
ACTUAL RESULTS	
Output Specification	[Record observed outputs, attach screen capture, and describe any discrepancy from the expected result.]

5.4 Non-Functional Requirements

Non-functional requirements describe quality attributes and are not satisfied by statements such as “the software is efficient” or “accurate”. Each requirement of Section 3.4 is therefore checked by a measurable criterion, and Table 5.6 summarizes the properties, the measures, the targets and the results. The purpose of the table is to compare each measured result with the target that was set in advance.

Table 5.6: Non-Functional Requirements, Measures and Results

Property	**Measure**	**Target**	**Result**
Accuracy	CER and WER on unseen writers	CER $\leq$ 8%, WER $\leq$ 20%	[measured]
Speed	End-of-word to candidates (ms)	Mean $\leq$ 300 ms; P95 $\leq$ 600 ms	[measured]
Usability	Attempts to first word; SUS score	$\leq$ 2 attempts; SUS $\geq$ 70	[measured]
Privacy	Network traffic from recognition	None carrying strokes or text	[measured]
Size	Installed storage (MB); peak RAM	$\leq$ 500 MB; within 3 GB	[measured]
Reliability	Crashes per 1,000 words; recovery time	0 crashes; $<$ 2 s	[measured]
Portability	Platforms passing functional tests	Android 13+, iOS 16+	[measured]

5.4.1 Accuracy Results

Accuracy is reported from Test Case 2 as the CER and WER of the full pipeline for unseen writers, together with the ablation results that show the effect of the RefinerGAN and of the language model. [Insert measured CER and WER and discuss which characters were confused most often, using the confusion matrix from the evaluation dashboard.]

5.4.2 Speed Results

Speed is reported from Test Case 4 as the mean and the 95th-percentile latency on the reference device. [Insert the measured values and state whether the target was met.]

5.4.3 Usability Results

Usability is reported from sessions with new users who wrote words after a short demonstration and then completed the System Usability Scale questionnaire. [Insert the number of participants, the attempts needed to write the first word and the average SUS score.]

5.4.4 Privacy Results

Privacy is reported from Test Case 5 by describing the traffic observed while the keyboard recognized handwriting. [State the result of the network monitoring test.]

5.4.5 Size and Resource Use Results

Size and resource use are reported as the installed storage of the application with all models and as the peak memory observed during recognition. [Insert the measured values.]

5.4.6 Reliability and Portability Results

Reliability is reported as the number of failures during 1,000 consecutive words and the recovery time after a forced restart, and portability as the list of platforms and devices on which the functional tests were passed. [Insert the measured values and the devices tested.]

Chapter 6

CONCLUSION

This chapter concludes the report. Section 6.1 summarizes the project, its key features and the results that were achieved, and Section 6.2 explains the limitations of the developed system and how they can be overcome in future work.

6.1 Conclusion

Urdu-Ink Mobile was developed to give Urdu speakers a keyboard in which they can write naturally on the screen of a phone and obtain editable Unicode text in any application. Starting from the pre-classification and wavelet pipeline of Safdar et al. [1], the project extended recognition from isolated half-forms to continuous cursive words, added a RefinerGAN that cleans noisy touchscreen strokes, enlarged the training data with a CycleGAN, replaced classical classifiers with a Transformer and BiLSTM-CTC recognizer, and corrected errors with an Urdu N-gram model and BERT-Urdu. The whole pipeline is optimized to run locally on the device, so that handwriting stays private and the keyboard works offline. The testing described in Chapter 5 showed a CER of [value]%, a WER of [value]% and an average latency of [value] ms on the reference device, and [state whether each target of Section 3.4 was met]. These results show that stroke refinement, word-level sequence modeling and language-model correction can together make handwriting a practical way of entering Urdu text on a phone, and they address the limitations of tablet-based, isolated-character and cloud-dependent systems identified in Chapter 2.

6.2 Limitations and Future Work

The system has several limitations. It is optimized for Urdu Nastaliq/Naskh and does not support other languages or decorative calligraphic styles without further training. Real-time performance depends on the processing power of the device, and it may lag on older entry-level phones that have no capable mobile GPU or NPU. Extremely unusual or very untidy handwriting that deviates strongly from standard ligature formations can still lower the accuracy. Finally, optional features such as translation add-ons need an internet connection, so they are not available in the offline core.

These limitations can be addressed in future work. The recognizer can be extended to other Perso-Arabic languages that share the script, such as Pashto and Sindhi, by retraining on new corpora. Model compression through quantization, pruning and knowledge distillation can reduce the demands placed on entry-level devices. A larger corpus with more writers of different ages and skill levels, together with continued domain-adversarial training, can improve writer independence. The recognition granularity can be extended from words to complete sentences and to free-form writing with a continuous language model, and personalized adaptation to the user's own handwriting can be added while still keeping all data on the device. Finally, the keyboard can be ported to further Linux-based mobile operating systems.

Appendix – I

SUPPLEMENTARY MATERIAL

A.1 Definitions of Terms

Half-form: the shape that a letter takes at a given position within a connected Urdu ligature (initial, medial or terminal) or in isolation. **Ligature:** a connected group of characters written without lifting the pen between joined letters; a word may contain several ligatures. **Nastaliq/Naskh:** two calligraphic styles of the Perso-Arabic script, Nastaliq being the traditional style of Urdu, with diagonal baselines. **Input Method Editor (IME):** the operating-system service through which a keyboard application provides text input to other applications. **CTC:** Connectionist Temporal Classification, a loss function and decoding method that maps unsegmented input sequences to label sequences.

A.2 Software Installation Guide

[Add step-by-step instructions for installing the keyboard on Android and iOS, enabling it as an input method and switching to it.]

A.3 User Manual

[Add screenshots and instructions for writing a word, selecting a candidate, deleting text and changing settings.]

A.4 Source Code Listings

[Add selected source code, such as the stroke file reader and the inference wrapper, or point to the code repository.]

REFERENCES

Bibliography

[1] Q. A. Safdar, K. U. Khan, and L. Peng. [Title of the paper]. *Scientia Iranica D*, vol. 29, no. 5, pp. 2419–2436, 2022.

[2] S. Naz et al. [Title of the paper]. *Pattern Recognition*, vol. 47, pp. 1229–1248, 2014.

[3] M. I. Razzak et al. [Title of the paper]. *Knowledge-Based Systems*, vol. 23, no. 8, pp. 914–923, 2010.

[4] G. A. Abandah and F. T. Jamour. Recognizing handwritten Arabic script through efficient skeleton-based grapheme segmentation algorithm. In *Proc. International Conference on Intelligent Systems Design and Applications (ISDA)*, 2010.

[5] A. Graves. *Supervised Sequence Labelling with Recurrent Neural Networks*. Ph.D. dissertation, Technische Universität München, 2008.

[6] [Complete reference for Hussain and Khan – spatial–temporal neural network for Urdu ligature recognition.]

[7] [Complete reference for Malik and Khan – holistic recognition of isolated Urdu characters.]

[8] [Complete reference for Shahzad et al. – geometric features for isolated Urdu character recognition.]

[9] I. Goodfellow et al. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[10] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2223–2232.

[11] A. Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proc. International Conference on Machine Learning (ICML)*, 2006, pp. 369–376.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[14] Y. Ganin et al. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.